P.PORTO

ESCOLA SUPERIOR DE TECNOLOGIA E GESTÃO

Validação, Interoperabilidade, Segurança e Documentação

Bruno Oliveira (bmo@estg.ipp.pt)

PEI Processamento Estruturado de Informação

Agenda

- Validação de Dados
- Interoperabilidade
- Segurança
- Documentação

Introdução

- Embora o JSON seja amplamente utilizado, o XML permanece profundamente enraizado em setores críticos como serviços financeiros, telecomunicações, saúde (HL7) e sistemas corporativos (SOAP).
- Uma API Node.js que não sabe processar XML é uma API que não consegue se integrar a uma porção substancial da infraestrutura de software global.

Validação de conteúdo e transformação em XML

- Ao contrário do JSON, em que o Express sabe manipular nativamente através de express.json(), o XML requer tratamento especial.
- O Express não analisa corpos de requisição XML por padrão.
- Para suportar XML, precisamos de uma estratégia de middleware que:
  - Ler o corpo da requisição como texto bruto (string).
  - Detectar o cabeçalho Content-Type: application/xml.
  - Utilizar o fast-xml-parser para converter essa string em um objeto JS.
  - Substituir o req.body pelo objeto convertido, permitindo que os validadores e controladores tratem a requisição como se fosse JSON nativo.

Validação de conteúdo e transformação em XML

```
const { XMLParser } = require('fast-xml-parser');
const express = require('express');

const app = express();
const PORT = 3000;

// Configuração do parser XML
const parser = new XMLParser({
  explicitArray: false,          // Evita criar arrays desnecessários
  ignoreAttributes: false,       // Mantém os atributos XML
  attributeNamePrefix: "@_",     // Prefixo para identificar atributos
  numberParseOptions: {
    hex: true,                   // Suporta números hexadecimais
    leadingZeros: false,         // Ignora zeros à esquerda
  }
});
```

Validação de conteúdo e transformação em XML

```
// Middleware para processar XML automaticamente
const xmlMiddleware = [
  // 1º passo: Lê o corpo da requisição como texto
  express.text({ type: ['application/xml', 'text/xml'] }),

  // 2º passo: Converte XML em JSON
  (req, res, next) => {
    // Verifica se a requisição é XML
    if (
      req.get('Content-Type') &&
      (req.get('Content-Type').includes('xml')) &&
      typeof req.body === 'string'
    ) {
      try {
        // Faz o parse do XML
        const parsed = parser.parse(req.body);

        // Substitui o XML pelo JSON no req.body
        req.body = parsed;
        next();
      } catch (err) {
        // Retorna erro se o XML for inválido
        return res.status(400).json({
          status: 'error',
          message: "Formato XML inválido ou malformado",
          details: err.message
        });
      }
    } else {
      // Se não for XML, continua normalmente
      next();
    }
  }
];
```

snappify.com

Validação de conteúdo e transformação em XML

```
// Aplica os middlewares
app.use(xmlMiddleware);  // Middleware de conversão XML
app.use(express.json()); // Middleware para JSON normal

// Rota de teste - POST /test-xml
// Recebe XML e retorna JSON com os dados convertidos
app.post('/test-xml', (req, res) => {
  res.json({
    status: 'success',
    message: 'XML convertido com sucesso!',
    receivedData: req.body,      // Dados já convertidos para JSON
    originalType: 'XML',
    convertedTo: 'JSON'
  });
});
```

snappify.com

Validação de conteúdo e transformação em XML

```
// Aplica os middlewares
app.use(xmlMiddleware);  // Middleware de conversão XML
app.use(express.json()); // Middleware para JSON normal

// Rota de teste - POST /test-xml
// Recebe XML e retorna JSON com os dados convertidos
app.post('/test-xml', (req, res) => {
  res.json({
    status: 'success',
    message: 'XML convertido com sucesso!',
    receivedData: req.body,      // Dados já convertidos para JSON
    originalType: 'XML',
    convertedTo: 'JSON'
  });
});
```

snappify.com

Validação de XML com XSD

- Validar XML em Node.js utilizando arquivos XSD é notoriamente difícil.
- As bibliotecas disponíveis muitas vezes dependem de bindings C++ (node-xerces) ou requerem a execução de binários Java externos, o que quebra a portabilidade da aplicação.
- Implementações puramente JavaScript de validadores XSD são frequentemente incompletas
- Vamos utilizar a biblioteca: xsd-schema-validator (https://www.npmjs.com/package/xsd-schema-validator?activeTab=readme). Requer o java disponível na linha de comandos.

Validação de XML com XSD

```
//(...)
const validator = require('xsd-schema-validator'); // Importa a biblioteca de validação XSD
const path = require('path'); // Importa o módulo para manipulação de caminhos de arquivos
const fs = require('fs'); // Importa o módulo de sistema de arquivos
// Middleware para processar XML automaticamente
const xmlMiddleware = [
  express.text({ type: ['application/xml', 'text/xml'] }),
  // 2º passo: Valida XSD e converte XML em JSON
  async (req, res, next) => { // Torna a função assíncrona para usar await
    // Verifica se a requisição é XML
    if (req.get('Content-Type') && (req.get('Content-Type').includes('xml')) && typeof req.body === 'string') {
      console.log('--> Recebida requisição XML. Iniciando processamento...'); // Log para debug

      // Usamos caminho relativo para evitar problemas com caracteres especiais no caminho absoluto
      const absoluteSchemaPath = path.join(__dirname, 'schemas', 'pessoa.xsd'); // Define o caminho absoluto do XSD
      const schemaPath = path.relative(process.cwd(), absoluteSchemaPath); // Converte para relativo para evitar bugs com acentos no Java

      console.log('--> Caminho relativo:', schemaPath);

      (...)
```

snappify.com

Validação de XML com XSD

```
      (...)
      try {
        // Valida o XML contra o XSD
        await validator.validateXML(req.body.trim(), schemaPath); // Executa a validação. .trim() remove espaços extras
        // Faz o parse do XML
        const parsed = parser.parse(req.body);
        req.body = parsed;
        next();

      } catch (err) {
        // Verifica se é erro de validação XSD
        if (err.valid === false || err.message.includes('invalid xml')) { // Verifica se o erro é de validação
           return res.status(400).json({
            (...)
            validationErrors: err.messages // Inclui as mensagens detalhadas do validador XSD
          });
        }

        // Outros erros (ex: parse do XML falhou depois da validação, ou erro interno)
        return res.status(400).json({
          status: 'error',
          message: "Erro no processamento do XML",
          details: err.message
        });
      }
    } else {
      next();
    }
  }
];
```

snappify.com

Autenticação

- Em arquiteturas RESTful, a ausência de estado (statelessness) é um princípio fundamental.
- O servidor não deve manter o estado da sessão do utilizador em memória entre requisições. Isso permite que a aplicação escale horizontalmente (adicionando mais servidores) sem se preocupar com a sincronização de sessões.
- A solução padrão da indústria para autenticação stateless é o JSON Web Token (JWT).

Anatomia e Segurança do JWT

- Um JWT é um meio compacto e seguro de URL para representar reivindicações (claims) entre duas partes.
- Um token consiste em três partes separadas por pontos (.):
  - Header (Cabeçalho): Especifica o algoritmo de assinatura (ex: HS256 ou RS256) e o tipo do token.
  - Payload (Carga Útil): Contém os dados do utilizador e metadados. Exemplos: sub (ID do utilizador), role (papel), iat (emitido em) e exp (expiração).
  - Signature (Assinatura): Uma hash criptográfico do Header e do Payload, gerado usando uma chave secreta que apenas o servidor possui.

Fluxo de Autenticação

- O ciclo de vida seguro de autenticação JWT em Express segue o padrão "Sign and Verify" (Assinar e Verificar) :
  - Login (Sign):
    - Cliente envia POST com credenciais
    - Servidor valida na base de dados (verificando hash de senha).
    - Se válido, servidor assina um token usando sua chave secreta privada.
    - Servidor devolve o token ao Cliente.
  - Acesso (Verify):
    - Cliente armazena o token.
    - Para rotas protegidas, o Cliente anexa o token no cabeçalho Authorization (Formato: Bearer <token>).
    - Middleware do Servidor intercepta, extrai o token e verifica a assinatura.
    - Se a assinatura for correta e o token não estiver expirado, o acesso é concedido.

Fluxo de Autenticação

```
(...)
const jwt = require('jsonwebtoken'); // Importa a biblioteca JWT
(...)
const SECRET_KEY = 'minha_chave_secreta_super_segura'; // Chave secreta para assinar o token
(...)
// Middleware de autenticação JWT
function authenticateToken(req, res, next) {
  // Procura o token no header Authorization (formato: "Bearer <token>")
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];

  if (token == null) {
    return res.status(401).json({ status: 'error', message: 'Token de autenticação não fornecido' });
  }

  jwt.verify(token, SECRET_KEY, (err, user) => {
    if (err) {
      return res.status(403).json({ status: 'error', message: 'Token inválido ou expirado' });
    }
    req.user = user; // guarda os dados do utilizador na requisição
    next();
  });
}
```

snappify.com

Fluxo de Autenticação

```
// Rota de Login - POST /login - Gera um token JWT para credenciais válidas
app.post('/login', (req, res) => {
  // Aceita credenciais via JSON ou XML (já convertido pelo middleware)
  // Estrutura esperada: { "username": "admin", "password": "password" }
  // Se vier do XML, pode estar dentro de um objeto raiz, ex: req.body.login.username

  let username, password;

  (...)

  // Validação simples (hardcoded para exemplo)
  if (username === 'admin' && password === 'password') {
    const user = { name: username };
    // Gera o token com validade de 1 hora
    const accessToken = jwt.sign(user, SECRET_KEY, { expiresIn: '1h' });

    res.json({
      status: 'success',
      message: 'Autenticação realizada com sucesso',
      accessToken: accessToken
    });
  } else {
    res.status(401).json({ status: 'error', message: 'Credenciais inválidas' });
  }
});
```

snappify.com

Fluxo de Autenticação

```
// Rota de teste - POST /test-xml (PROTEGIDA)
app.post('/test-xml', authenticateToken, (req, res) => {
  res.json({
    status: 'success',
    message: 'XML convertido com sucesso!',
    user: req.user,              // Dados do utilizador extraídos do token
    receivedData: req.body,
    originalType: 'XML',
    convertedTo: 'JSON'
  });
});
```

snappify.com

Documentação com Swagger (OpenAPI)

- No passado, a documentação era um arquivo de texto estático ou uma wiki.
- Inevitavelmente, o código muda, a wiki era esquecida e a documentação tornava-se obsoleta, enganosa e inútil.
- A OpenAPI Specification resolve isso tratando a documentação como código.
- Definimos a estrutura da API junto com a lógica, permitindo a geração de interfaces interativas para testar os endpoints em tempo real.

Documentação com Swagger (OpenAPI)

- Utilizamos duas bibliotecas essenciais no ecossistema Express:
  - swagger-jsdoc: Procura no código fonte comentários JSDoc especialmente formatados (anotações YAML) e gera a especificação JSON da OpenAPI.
  - swagger-ui-express: Consome essa especificação JSON e renderiza uma página web e interativa (o Swagger UI) servida diretamente pelo Express.

Documentação com Swagger (OpenAPI)

```
(...)
const swaggerUi = require('swagger-ui-express');
const swaggerJsdoc = require('swagger-jsdoc');

(...)
// Configuração do Swagger
const swaggerOptions = {
  definition: {
    openapi: '3.0.0',
    info: {
      title: 'Express XML & JWT API',
      version: '1.0.0',
      description: 'API de exemplo com suporte a XML, validação XSD e autenticação JWT',
    },
    servers: [
      {
        url: `http://localhost:${PORT}`,
      },
    ],
    components: {
      securitySchemes: {
        bearerAuth: {
          type: 'http',
          scheme: 'bearer',
          bearerFormat: 'JWT',
        },
      },
    },
  },
  // Caminho para os arquivos que contêm anotações do Swagger
  // Ajustar o caminho conforme necessário dependendo de onde   executa o servidor
  apis: [path.join(__dirname, 'app.js')],
};
(...)
```

snappify.com

Documentação com Swagger (OpenAPI)

```
(...)
const swaggerDocs = swaggerJsdoc(swaggerOptions);
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerDocs));

(...)
// Rota de Login - POST /login
/**
 * @swagger
 * /login:
 *   post:
 *     summary: Autentica o usuário e retorna um token JWT
 *     tags: [Auth]
 *     requestBody:
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             type: object
 *             properties:
 *               username:
 *                 type: string
 *                 example: admin
 *               password:
 *                 type: string
 *                 example: password
 *         (...)
 *       401:
 *         description: Credenciais inválidas
 */
app.post('/login', (req, res) => { (...)
```

snappify.com

Módulos

Em Node.js, cada ficheiro é tratado como um módulo separado.

Um módulo é uma unidade de código encapsulada que pode conter funções, objetos ou variáveis.

O código dentro de um módulo é privado (não visível externamente) a menos que seja explicitamente exportado. Isso ajuda a organizar o código, evitar conflitos de nomes e promover a reutilização.

Módulos

Para tornar as funcionalidades de um módulo disponíveis para outros ficheiros, usamos o sistema de exportação:

- module.exports: É o objeto que é devolvido quando se utiliza require('./arquivo'). Pode ser atribuído a um objeto, uma função, ou qualquer valor.
  - *Exemplo:* module.exports = minhaFuncao; (exporta uma única função)
  - *Exemplo:* module.exports = { func1, func2 }; (exporta um objeto)
- exports: É uma referência auxiliar para module.exports. É normalmente utilizado para adicionar propriedades ao objeto exportado.
  - *Exemplo:* exports.login = ... é o mesmo que module.exports.login = ...

Módulos

Reorganizando o código apresentado anteriormente, obtemos uma estrutura mais modular escalável:

- **config**: Configurações do projeto (Swagger, Autenticação).
- **controllers**: Lógica de controlo das requisições (Auth, XML).
- **middlewares**: Middlewares do Express (Auth, XML Parsing/Validation).
- **routes**: Definição das rotas da API.
- **schemas**: Schemas XSD para validação.
- **app.js**: Ponto de entrada da aplicação.

Módulos

Auth.js

```
// module.exports é usado para exportar o objeto de configuração
//para que possa ser utilizado noutros ficheiros com require()
module.exports = {
  SECRET_KEY: 'minha_chave_secreta_super_segura',
  expiresIn: '1h'
};
```

swagger.js

```
(...)
const swaggerDocs = swaggerJsdoc(options);
// module.exports exporta a configuração gerada do Swagger
//para ser utilizado no ficheiro principal (app.js)
module.exports = swaggerDocs;
```

Módulos

```
const swaggerDocs = require('./config/swagger');
const xmlMiddleware = require('./middlewares/xmlMiddleware');
const authRoutes = require('./routes/authRoutes');
const xmlRoutes = require('./routes/xmlRoutes');
const indexRoutes = require('./routes/index');

(...)

// Documentação Swagger
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerDocs));

// Middlewares Globais
app.use(xmlMiddleware);
app.use(express.json());

// Rotas
app.use('/', indexRoutes);
app.use('/', authRoutes);
app.use('/', xmlRoutes);

(...)
```

snappify.com

Referências

- Referências Web:
  - https://expressjs.com/en/resources/middleware/body-parser.html
  - https://nodejs.org/api/modules.html
  - https://auth0.com/docs/secure/tokens/json-web-tokens
  - https://johnnyreilly.com/xml-read-and-write-with-node-js
- Livro:
  - Lauret A., The Design of Web APIs, Manning, 2019;

P.PORTO

ESCOLA SUPERIOR DE TECNOLOGIA E GESTÃO

Validação, Interoperabilidade, Segurança e Documentação

Bruno Oliveira (bmo@estg.ipp.pt)

PEI Processamento Estruturado de Informação